

— Databanken

Hoofdstuk 4: Gegevens selecteren uit een databank

L. Espeel



— SELECT-query

Dit is de algemene vorm van een SELECT-query met rechts de effectieve volgorde van uitvoer:

SELECT DISTINCT <veldna(a)m(en)>	5, 6
FROM <tabelnaam>	1
JOIN <tabelnaam> ON <id> = <id>	1 (kan 1 of meerdere joins bevatten)
WHERE <conditie(s)>	2 (kan naast filtering eventueel subqueries bevatten)
GROUP BY <veldna(a)m(en)>	3
HAVING <conditie(s)>	4
ORDER BY <veldna(a)m(en)>	7
LIMIT <offset>,<aantal>;	8

- De veldnamen worden gescheiden door komma's, om alle velden te selecteren kan er gebruik gemaakt worden van *
- Importeer voor dit hoofdstuk de databank *jeugdbeweging.sql* in MySQL Workbench.

— Velden selecteren (projectie)

- Selecteer alle velden:
`SELECT *`
`FROM activiteiten;`
- Selecteer enkel de velden *naam* en *geboortedatum*:
`SELECT naam, geboortedatum`
`FROM leden;`

3

— Records filteren (selectie) ⁽¹⁾

- `SELECT naam, adres, postnummer, gemeente`
`FROM leden`
`WHERE ismeisje = 1;`
- `SELECT *`
`FROM activiteiten`
`WHERE omschrijving = "Vakantiekamp";`

Opmerkingen:

- SQL is standaard **hoofdletterongevoelig** voor vergelijkingen op strings
- Strings worden aangegeven tussen **dubbele "** of **enkele '**
 - Ook **datums** en **tijdstippen** moet je als strings beschouwen in de vergelijkingen
- Vergelijkingen op een exacte waarde is met een **enkele =**
- Andere veelgebruikte **operatoren**:
 - **<>** (of **!=**)
 - **>** en **<**
 - **>=** en **<=**

4

— Records filteren (selectie) (2)

Logische operatoren

Conditioes kan je gaan uitbreiden met deze logische operatoren:

Logische operatoren		
AND	BETWEEN	ALL
OR	IN	ANY
NOT	LIKE	EXISTS

Opmerking:

- all, any en exists worden gebruikt in subqueries (zie later).

5

— Records filteren (selectie) (3)

AND, OR en NOT

- ```
SELECT naam, geboortedatum
FROM leden
WHERE gemeente = "Veurne" AND ismeisje = 0;
```
- ```
SELECT *
FROM leden
WHERE gemeente = "Waregem" OR gemeente = "Tielt";
```
- ```
SELECT omschrijving, datumbijdstip
FROM activiteiten
WHERE NOT omschrijving = "Halloweentocht";
```

6

## — Records filteren (selectie) <sup>(4)</sup>

### *BETWEEN*

- Grenzen zijn inclusief:
  - `getal BETWEEN 2 AND 7`
  - `getal >= 2 AND getal <= 7`
- `SELECT naam, adres, postnummer, gemeente`  
`FROM leden`  
`WHERE postnummer BETWEEN 8000 AND 9000;`
- `SELECT *`  
`FROM activiteiten`  
`WHERE datumtijdstip BETWEEN "2015-01-01 00:00:00" AND "2015-12-31 23:59:59";`

7

## — Records filteren (selectie) <sup>(5)</sup>

### *IN*

- Dit vervangt een opeenvolging van OR-operatoren:
  - `getal = 1 OR getal = 2 OR getal = 3`
  - `getal IN (1, 2, 3)`
- `SELECT naam, ismeisje`  
`FROM leden`  
`WHERE ismeisje = 1`  
`AND gemeente IN ("Tielt", "Veurne", "Waregem");`

8

## — Records filteren (selectie) <sup>(6)</sup>

*LIKE*

- Dit zoekt op een deel van een woord op basis van jokertekens:
  - naam LIKE "Marie%"
- Voorbeelden:
  - K%
  - %oe
  - %en%
  - \_a%

### Jokertekens

|   |                         |
|---|-------------------------|
| % | willekeurige tekenreeks |
| _ | willekeurig teken       |

9

## — Records filteren (selectie) <sup>(7)</sup>

*LIKE met jokertekens*

- ```
SELECT *  
FROM leden  
WHERE naam LIKE "%Van%";
```
- ```
SELECT naam, adres, postnummer, gemeente
FROM leden
WHERE gemeente LIKE "_i%";
```
- ```
SELECT naam, ismeisje  
FROM leden  
WHERE naam LIKE "__n%";
```

10

— Records filteren (selectie) ⁽⁸⁾

Reguliere expressies

- Probleem met % en _
 - Deze zijn ontoereikend om bv. te zoeken op namen die beginnen met een klinker.
- Oplossing:
 - Gebruik maken van reguliere expressies die een patroon zijn voor complexere zoekacties.
- ```
SELECT naam
FROM leden
WHERE naam REGEXP "^[aeiou]";
```

11

## — Records filteren (selectie) <sup>(9)</sup>

### *Reguliere expressies*

Betekenis symbolen:

|                |                                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------------|
| <b>^</b>       | match the beginning of a string                                                                              |
| <b>\$</b>      | match the end of a string                                                                                    |
| <b>.</b>       | match any character                                                                                          |
| <b>a*</b>      | match any sequence of <b>zero or more</b> <i>a</i> characters                                                |
| <b>a+</b>      | match any sequence of <b>one or more</b> <i>a</i> characters                                                 |
| <b>a?</b>      | match either <b>zero or one</b> <i>a</i> characters                                                          |
| <b>[a-dX]</b>  | matches <b>any character</b> that is <b>either</b> <i>a</i> , <i>b</i> , <i>c</i> , <i>d</i> or <i>X</i>     |
| <b>[^a-dX]</b> | matches <b>any character</b> that is <b>not either</b> <i>a</i> , <i>b</i> , <i>c</i> , <i>d</i> or <i>X</i> |

12

## — Records filteren (selectie) <sup>(10)</sup>

*Reguliere expressies*

Enkele voorbeelden:

| reguliere expressie         | voldoet              | voldoet niet           |
|-----------------------------|----------------------|------------------------|
| <code>[aeoui]</code>        | ai, aap, mes, wei    | ly, ssst, krpk         |
| <code>[^aeoui]</code>       | ly, ssst, krpk       | ai, aap, mes, wei      |
| <code>[a-d]</code>          | schop, as, weldra    | yoghurt, zes           |
| <code>[aeoui][aeoui]</code> | pauw, aas, prooi     | wens, arcade, kriskras |
| <code>^[aeoui0-9]</code>    | agenda, 8tzaam, 44   | z3s, w8, vrees         |
| <code>[aeoui0-9]\$</code>   | agenda, 44, w8       | z3s, vrees, 8tzaam     |
| <code>ka.s</code>           | kaas, herkansing     | kas                    |
| <code>ka.*s</code>          | kas, kaas, kamikase  | aks, sake              |
| <code>ka*s</code>           | ks, kas, kaas, kaaas | kees                   |
| <code>ka+s</code>           | kas, kaas, kaaas     | ks, kees               |
| <code>ka?s</code>           | ks, kas              | kaas                   |

13

## — Records filteren (selectie) <sup>(11)</sup>

*Reguliere expressies*

Probeer eens volgende websites uit om reguliere expressies te leren, gebruik geen AI:

- <https://regexlearn.com/learn/regex101>: interactief leren
- <https://regex101.com>: een reguliere expressie ontleden/opmaken en uittesten met een waarde

14

## — Records filteren (selectie) <sup>(12)</sup>

*Reguliere expressies in SQL-queries*

Enkele SQL-query voorbeelden:

- `SELECT * FROM leden WHERE naam REGEXP "^[^A-C]";`
  - Toont alle leden waarvan de naam niet begint met A, B of C
- `SELECT * FROM leden WHERE naam REGEXP "^on";`
  - Toont alle leden waarvan de naam begint met "on"
- `SELECT * FROM leden WHERE naam REGEXP "be | ae";`
  - Toont alle leden die "be" of "ae" bevatten
- `SELECT * FROM leden WHERE naam REGEXP "[b-f].[a]";`
  - Toont alle leden die b, c, d, e of f bevatten gevolgd door een enkele letter en daarna een a

15

## — Records sorteren

Sorteren:

- Oplopend (a → z): **ASC** (= standaard, ASC kan ook weggelaten worden)
- Aflopend (z → a): **DESC**
- `SELECT naam`  
`FROM leden`  
`WHERE ismeisje = 1`  
`ORDER BY naam ASC;`
- `SELECT omschrijving, datumbijeenkomst`  
`FROM activiteiten`  
`ORDER BY omschrijving ASC, datumbijeenkomst DESC;`

16



## — Records limiteren

Hiermee kan je een subset van records nemen.

- **LIMIT 0, 5**
  - Selecteert de eerste 5 records, 0 mag hier zelfs weggelaten worden
- **LIMIT 9, 3**
  - Selecteert de tiende, elfde en twaalfde record
- Is handig in combinatie met een **ORDER BY**:  

```
SELECT naam
FROM leden
WHERE gemeente = "Waregem"
ORDER BY geboortedatum DESC
LIMIT 3;
```

17

## — Unieke records

Met **DISTINCT** kan je dubbele resultaten vermijden.

- ```
SELECT DISTINCT omschrijving  
FROM activiteiten;
```
- ```
SELECT DISTINCT adres
FROM leden
ORDER BY adres;
```
- ```
SELECT DISTINCT adres, postnummer  
FROM leden  
ORDER BY adres;
```

18